

International Islamic University Islamabad

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering



MACHINE LEARNING LAB LAB MANUAL

Name of Student:[Bushra Nazir Ahmed](#).....

Registration No.:[1071-F22](#).....

Submitted To:[Engr.Asad](#).....

INTERNATIONAL ISLAMIC UNIVERSITY ISLAMABAD
FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRICAL ENGINEERING

**Artificial Intelligence & Machine Learning
LAB**

Experiment List

Experiment No.	Experiment Title
1	Hands-on Python Programming for Artificial Intelligence & Machine Learning
2	Implementing Numerical Data Operations Using NumPy
3	Performing Data Analysis and Visualizing Results Using Pandas and Matplotlib
4	Raspberry Pi Setup for AI and Machine Learning
5	Physical Computing with Raspberry PI
6	Design and Implementation of AI Search Algorithms for Pathfinding and Game Decision Making
7	Implementation of Regression Techniques using scikitlearn Library
8	Deploying a Regression Model on Raspberry Pi with Live Sensor Data
9	Data-Driven Classification using Logistic Regression and KNN.
10	Implementation of Logistic Regression and KNN on Raspberry Pi
11	Design and Implementation of Artificial Neural Networks (ANN).
12	Advanced Classification Using Decision Tree, Random Forest, and SVM
13	Edge ML Deployment: Running DT, RF and SVM classifier on Raspberry Pi with Real Time Sensor Interface
14	Implementing Convolutional Neural Network (CNN) for Edge-Based Mask Detection and Face Recognition

International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical Engineering

Artificial Intelligence & Machine Learning LAB

Open Ended Lab Evaluation

Group No.	Student No. 1	Name	Bushra Nazir	Reg. No.	1071-F22
	Student No. 2	Name		Reg. No.	
	Student No. 3	Name		Reg. No.	
	Student No. 4	Name		Reg. No.	
	Student No. 5	Name		Reg. No.	

S. No.	Criteria	Allocated Percentage	Level 1 (0%)	Level 2 (25%)	Level 3 (50%)	Level 4 (75%)	Level 5 (100%)	S1 Marks Obtained	S2 Marks Obtained	S3 Marks Obtained	S4 Marks Obtained	S5 Marks Obtained
(A) <u>PSYCHOMOTOR</u> (To be judged in the lab during experiment)												
1	Practical Implementation OR Connectivity/Arrangement of Equipment	5	Absent	Degree of errors in applying procedural knowledge or Degree of errors in arrangement of equipment:								
			0	With several critical errors, Imcomplete and Not Neat	With few errors, Incomplete and not Neat	With some errors, Complete but not Neat	Without errors, Complete and Neat					
				1.25	2.5	3.75	5					
2	Use of Equipment OR Use of Simulation/Programming Tool	2	Absent	Use of tools, equipment and materials with:								
			0	Limited Competence	Some Competence	Considerable Competence	Good Competence					
				0.5	1	1.5	2					
3	Safety (Optional)	0	Absent	Degree of reminders to follow safety procedure with:								
			0	Rarely Follow Safety Rules	Needs Reminders	Needs minor Reminders	Follows Safety Rules					
				0	0	0	0					
SUB-TOTAL (PT)		7	SUB-TOTAL MARKS OBTAINED (PO)									
(B) <u>COGNITIVE</u> (To be judged on the copy of experiment submitted)												
4	Data Record, Analysis and Evaluation OR Algorithm Design	1	Absent	Data record, Analysis and Evaluation or Designed Algorithm is:								
			0	Incorrect /Incomplete	Complete with some errors	Complete with few errors	Complete and Accurate					
				0.25	0.5	0.75	1					
5	Procedural Awareness (Optional)	0	Absent	Procedural awareness of the completed task is:								
			0	Inappropriate	Appropriate with some errors	Appropriate with few errors	Appropriate					
				0	0	0	0					
SUB-TOTAL (CT)		1	SUB-TOTAL MARKS OBTAINED (CO)									
(C) <u>AFFECTIVE</u> (To be judged in the lab during experiment)												
6	Level of Participation & Attitude to Achieve Individual/Group Goals	2	Absent	Degree of positive interaction as an individual or within a group:								
			0	Rare interaction	Some interaction	Acceptable interaction	Good interaction					
				0.5	1	1.5	2					
SUB-TOTAL (AT)		2	SUB-TOTAL MARKS OBTAINED (AO)									

Total Marks (PT + CT + AT) :

10	10	10	10	10
----	----	----	----	----

Marks Obtained (PO + CO + AO) :

--	--	--	--	--

Instructor

International Islamic University Islamabad

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering



AI & Machine Learning Lab

Experiment No. 1: Hands-on Python Programming for Artificial Intelligence & Machine Learning

Name of Student: ..[Bushra Nazir](#).....

Registration No.:[1071-F22](#).....

Date of Experiment:[22-2-2026](#).....

Submitted To:[Engr.Asad](#).....

Objectives:

- Understand Python's importance in AI/ML and execute basic scripts using Jupyter Notebook or Google Colab.
- Utilize Python data types, data structures (arrays, lists, tuples, dictionaries, Data Frames), conditional statements, loops, and functions for structured programming.
- Perform file handling and data manipulation using Pandas, including reading/writing files, filtering, sorting, and summarizing datasets.

Equipment & Tools Required

Hardware:

- Personal Computer/Laptop with a minimum of 4GB RAM (8GB recommended)

Software & Tools:

- Python (Latest Version - 3.x)
- Jupyter Notebook (via Anaconda) or Google Colab
- Internet Access (*fordownloadingpackagesandusing Google Colab if needed*)

Understanding AI, ML, and Data Science

In the modern digital era, terms like Artificial Intelligence (AI), Machine Learning (ML), and Data Science are often used interchangeably, but they have distinct meanings and applications.

- **Artificial Intelligence (AI)** is the broadest field, referring to any system that mimics human intelligence. It includes rule-based systems, expert systems, robotics, and machine learning.
- **Machine Learning (ML)** is a subset of AI that focuses on developing algorithms that enable computers to learn from data and make predictions or decisions without being explicitly programmed.
- **Data Science** is a multidisciplinary field that extracts meaningful insights from data using statistics, ML, and data visualization techniques. It acts as the bridge between raw data and AI-powered applications.

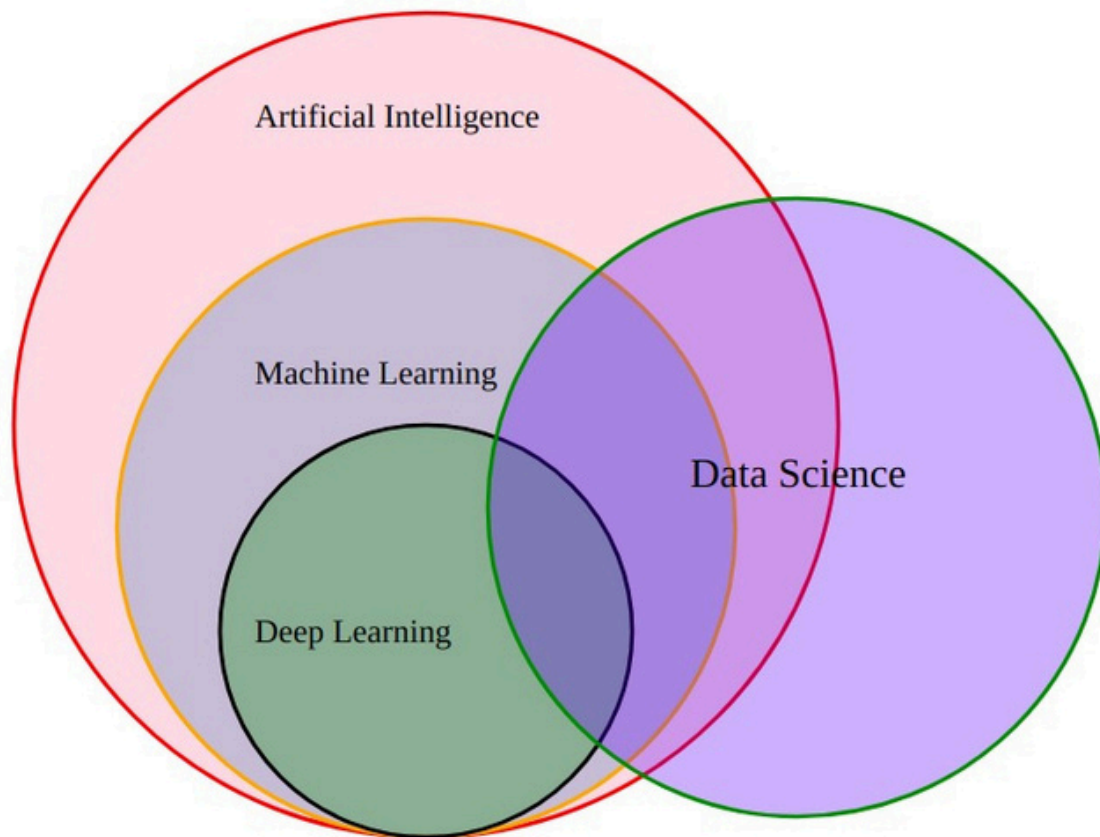
Example: Self-Driving Cars

Imagine a self-driving car navigating through a city:

- **AI** enables the car to make real-time driving decisions, such as stopping at red lights or changing lanes.
- **ML** helps the car improve its performance by learning from past driving data, recognizing objects, and predicting pedestrian movements.

- **Data Science** processes vast amounts of sensor data, GPS logs, and road conditions to optimize the car's route and performance.

This lab manual introduces Python as a key tool in AI, ML, and Data Science, helping students build intelligent systems that can analyze data, recognize patterns, and make **predictions**



Introduction to Python

Python is a high-level, interpreted programming language known for its simplicity, readability, and versatility. Developed by Guido van Rossum and first released in 1991, Python has grown to become one of the most popular programming languages in the world. Its easy-to-read syntax and extensive libraries make it an excellent choice for beginners and professionals.

Python is widely used in various domains, including web development, data science, artificial intelligence, machine learning, automation, and embedded systems. It supports multiple programming paradigms, such as procedural, object-oriented, and functional programming, making it highly adaptable for different applications.

Key features of Python include:

- ✓ **Simple and Readable Syntax** – Python code is easy to understand and write, making development faster.
- ✓ **Interpreted Language** – Python executes code line by line, which simplifies debugging.
- ✓ **Extensive Libraries and Frameworks** – Python offers a vast collection of libraries like NumPy, Pandas, TensorFlow, Django, and Flask.
- ✓ **Cross-Platform Compatibility** – Python runs on Windows, macOS, and Linux, allowing flexibility in development.
- ✓ **Community Support** – Python has a large and active community, ensuring continuous improvement and support.

Because of its ease of use and powerful capabilities, Python is a preferred language for both beginners and experienced developers working on small scripts or large-scale applications.

Setting Up Python

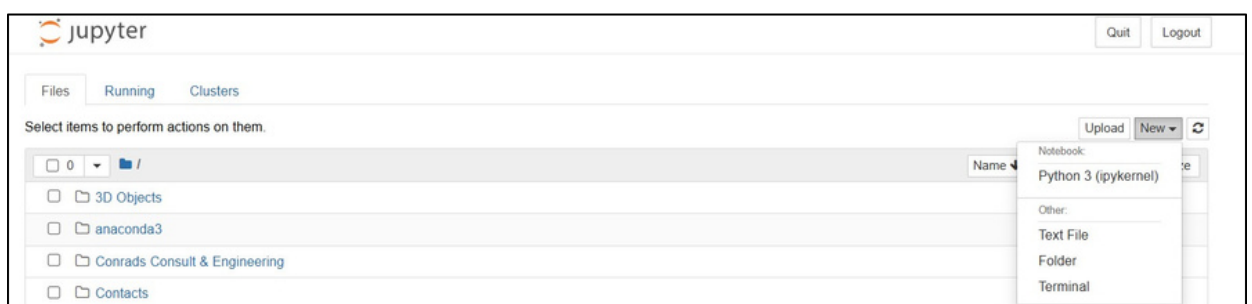
To work with Python for AI and ML, we need an environment where we can write, execute, and manage Python code efficiently. Below are the most commonly used setups:

1. Installing Python via Anaconda (Recommended)

Anaconda is a free, open-source distribution of Python that comes pre-installed with essential AI/ML libraries and tools like Jupyter Notebook.

Steps to Install Anaconda:

1. Download Anaconda from <https://www.anaconda.com/download> (choose the version for your operating system).
2. Follow the installation instructions and ensure you add Anaconda to your system's PATH (optional but recommended).
3. Verify the installation by opening the terminal (or Anaconda Prompt)



and typing:

python --version

The command shows the version of python installed.

```
PS C:\Users\ASAD EHSAN> python --version
Python 3.10.9
```

4. Launch Jupyter Notebook from the Anaconda Navigator or by running:

```
jupyter notebook
```

2. Using Jupyter Notebook

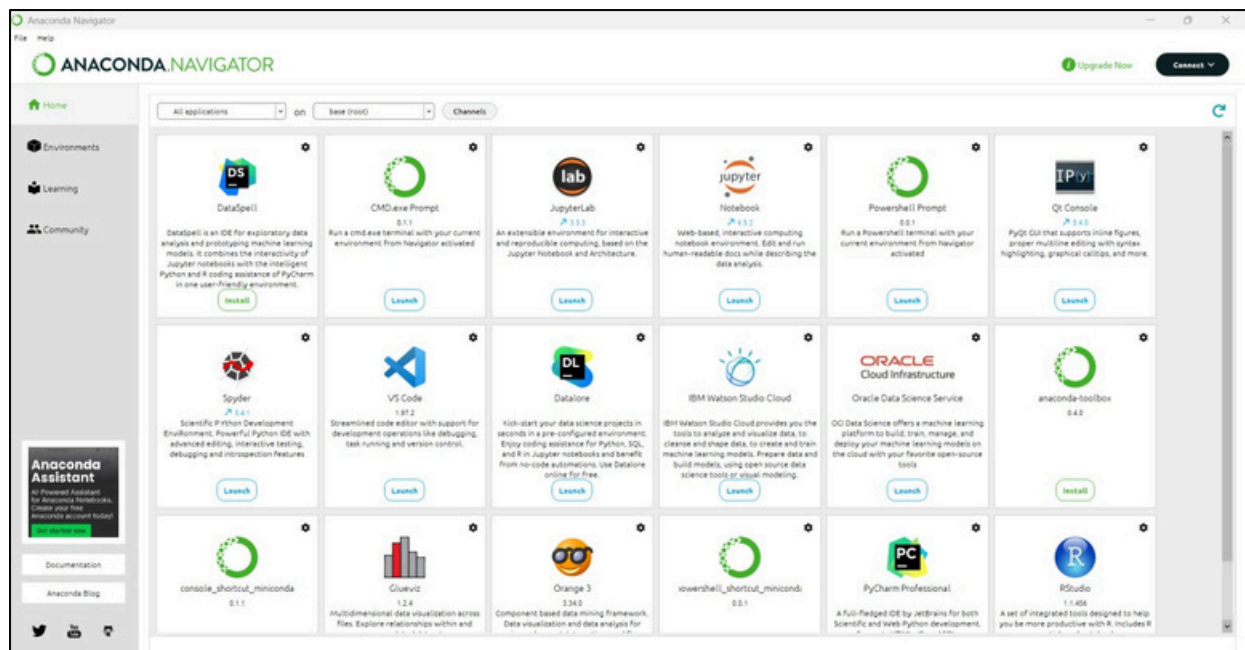
Jupyter Notebook is an interactive web-based Python environment widely used for AI and ML because it allows users to write code, visualize outputs, and document work in a single interface.

Key Features of Jupyter Notebook:

- Supports step-by-step execution of code
- Allows integration of text, images, and equations
- Facilitates data visualization
- Saves work in a shareable .ipynb format

To create a new notebook:

1. Open **Anaconda Navigator** and launch **Jupyter Notebook**, or run:
2. Click on **New** → **Python 3** to open a new notebook.
3. Write and execute Python code in individual cells.



3. Jupyter Lab

JupyterLab, included in Anaconda, is an advanced, flexible interface for Jupyter Notebooks, offering a multi-tab layout, integrated file browser, and support for text editors and terminals. Compared to Jupyter Notebook, JupyterLab provides a **more** interactive and modular environment, allowing users to open multiple notebooks, scripts, and visualizations side by side, making it ideal for AI/ML workflows.

4. Google Colab

Google Colaboratory (Colab) is a free cloud-based Jupyter Notebook that requires no setup and provides access to GPUs for deep learning.

Using Google Colab:

1. Open Google Colab.
2. Sign in with your Google account.
3. Click New Notebook to start coding.
4. Write Python code and execute it in the provided cells.

Why Choose Google Colab?

- No installation required
- Free access to cloud GPUs
- Seamless integration with Google Drive

Learning Python Language

1. Python Syntax

Python syntax refers to the set of rules that define how Python programs are written and interpreted. Python is known for its simplicity and readability, making it an ideal language for beginners in Machine Learning (ML) and Artificial Intelligence (AI).

Basic Syntax Rules

- Python uses indentation (not braces) to define code blocks.
- Statements are written line by line (no semicolons; needed).
- Comments start with # and are ignored by the interpreter.
- Variables are dynamically typed (no need to declare their type).

Indentation refers to the spaces at the beginning of a code line. Whereas in other programming languages, the indentation in code is for readability only, the indentation in Python is very important. Python uses indentation to indicate a block of code.

<pre>if x > 5: print("x is greater than 5")</pre>	<pre>if x > 5: print("x is greater than 5")</pre>
---	---

The number of spaces is up to you as a programmer, the most common use is four, but it has to be at least one

2. Python Variable

A **variable** in Python is a container for storing data. Unlike other programming languages, Python does not require explicit variable declarations; variables are created automatically when a value is assigned.

Example

```
x = 10 # Integer
y = 3.14 # Float
name = "AI" # String
is_active = True # Boolean
```

Python is dynamically typed, meaning you don't need to specify the data type. It is determined automatically based on the assigned value.

Variable Naming Rules

- Can contain letters(a-z, A-Z), digits (0-9), and underscores (_) only.
- Cannot start with a digit (e.g., 2name is invalid).
- Cannot use Python keywords (e.g., if, while, def, etc.).
- Case-sensitive (Age and age are different variables).

Valid Variables

```
my_var = 42
_name = "Python"
temperature_in_Celsius = 25.5
```

Invalid Variables

```
2name = "AI" # Invalid: starts with a number
class = "ML" # Invalid: 'class' is a keyword
price$ = 100 # Invalid: special characters not allowed
```

Multiple assignments

Python allows multiple assignments in a single statement

```
a, b, c = 10, 20, 30
print(a, b, c) # Outputs: 10 20 30
```



```
10 20 30
```

You can also assign same value to multiple variables.

```
x = y = z = "AI"
print(x, y, z) # Outputs: AI AI AI
```



```
AI AI AI
```

To check a variable's type, use the `type()` function:

```
x = 100          # int
y = 3.14         # float
name = "ML"      # str
status = True    # bool
data = [1, 2, 3] # list
```



```
print(type(x))   # Outputs: <class 'int'>
print(type(name)) # Outputs: <class 'str'>
```



```
<class 'int'>
<class 'str'>
```

Deleting a variable

```
x = 5
del x
#print(x) # This will cause an error because x no longer exists
```

```
print(x)

-----
NameError                                Traceback (most recent call last)
<ipython-input-15-e3d6ebd69964> in <cell line: 0>()
----> 1 print(x)  # This will cause an error because x no longer exists

NameError: name 'x' is not defined
```

Variables are fundamental in Python programming and are widely used in Machine Learning (ML) and Artificial Intelligence (AI) for storing data, model parameters, and **results**. In the next section, we will explore Python data types in detail.

3. Python Data Types

In Python, every variable has a specific data type that determines the kind of values it can store and the operations that can be performed on it. Python provides several built-in data types, which can be broadly categorized into:

1. Numeric Types
2. Sequence Types
3. Set Types
4. Mapping Types
5. Boolean Type
6. Binary Types

1. Numeric Types

Numeric data types store numbers and support arithmetic operations.

- **Integer (int)** – Whole numbers (positive or negative).
- **Floating Point (float)** – Decimal numbers.
- **Complex (complex)** – Numbers with a real and imaginary part.

```
a = 10      # Integer
b = 3.14    # Float
c = 2 + 3j   # Complex number

print(type(a))  # Output: <class 'int'>
print(type(b))  # Output: <class 'float'>
print(type(c))  # Output: <class 'complex'>

<class 'int'>
<class 'float'>
<class 'complex'>
```

2. Sequence Types

Sequence types store ordered collections of items.

- **String (str)** – A sequence of characters enclosed in quotes.
- **List (list)** – An ordered, mutable (changeable) collection.
- **Tuple (tuple)** – An ordered, immutable (unchangeable) collection.

```
# String
text = "Machine Learning"
print(type(text)) # <class 'str'>

# List (Mutable)
numbers = [1, 2, 3, 4]
numbers.append(5) # Adding an element
print(numbers) # Output: [1, 2, 3, 4, 5]

# Tuple (Immutable)
coordinates = (10, 20, 30)
print(coordinates[0]) # Output: 10
```

3. Set Types

Sets store **unique, unordered** collections of elements.

- **Set (set)** – A collection of unique, unordered elements.
- **FrozenSet (frozenset)** – A set that is immutable.

```
my_set = {1, 2, 3, 3, 4} # Duplicates are removed
print(my_set) # Output: {1, 2, 3, 4}

frozen = frozenset(my_set) # Immutable set

{1, 2, 3, 4}
```

4. Mapping Type (Dictionary)

Dictionaries store **key-value pairs**, allowing fast lookups

```
student = {"name": "Alice", "age": 21, "course": "AI"}
print(student["name"]) # Output: Alice

Alice
```

5. Boolean Type

Boolean (bool) values represent **True** or **False**.

```
x = True
y = False
print(type(x)) # Output: <class 'bool'>

<class 'bool'>
```

6. Binary Types

Python supports binarydata for handling files, images, etc.

- **Bytes (bytes)** –Immutable sequence of bytes.
- **Bytearray (bytearray)** – Mutable sequence of bytes.
- **MemoryView (memoryview)** – Provides memory-efficient access to binary data.

```
byte_data = b"AI" # Bytes
print(type(byte_data)) # Output: <class 'bytes'>

mutable_bytes = bytearray([65, 66, 67]) # Bytearray
mutable_bytes[0] = 68 # Modify first byte
print(mutable_bytes) # Output: bytearray(b'DBC')

<class 'bytes'>
bytearray(b'DBC')
```

Type Checking and Conversion

Python allows checking data typesand converting between them:

```
x = 10
print(type(x)) # Output: <class 'int'>

# Type conversion
y = float(x) # Convert int to float
z = str(x)   # Convert int to string
print(y, z)  # Output: 10.0 '10'

<class 'int'>
10.0 10
```

4. Data Structures in Python

A data structure is a way of organizing and storing data efficiently. Python provides built-in data structures that help manage and manipulate data in Machine Learning (ML) and Artificial Intelligence (AI) applications. The primary data structures in Python are:

1. Lists (Dynamic arrays)
2. Tuples (Immutable sequences)
3. Sets (Unordered collections of unique elements)
4. Dictionaries (Key-value pairs)

1. Lists (list)

A list is an ordered, mutable collection of items. It can hold multiple data types and supports various operations.

Creating list

```
# List of integers
numbers = [10, 20, 30, 40]

# List with mixed data types
data = ["AI", 3.14, True]

# Empty list
empty_list = []
```

List operations

```
# Accessing elements (indexing starts from 0)
print(numbers[1]) # Output: 20

# Slicing a list
print(numbers[:2]) # Output: [10, 20]

# Adding elements
numbers.append(50) # Adds 50 at the end

# Removing elements
numbers.remove(30) # Removes first occurrence of 30

# Sorting a list
numbers.sort() # Sorts in ascending order

20
[10, 20]
```

Tuples (tuple)

A tuple is similar to a list but immutable (cannot be modified after creation).

Creating a Tuple

```
coordinates = (10, 20, 30)
single_value_tuple = (42,) # A comma is required for a single-element tuple
```

Tuple Operations

```
[29] print(coordinates[1]) # Output: 20
      print(len(coordinates)) # Output: 3
```

```
⇒ 20
   3
```

Sets (set)

A set is an unordered collection of unique elements.

Creating a set

```
my_set = {1, 2, 3, 3, 4} # Duplicates are removed
print(my_set) # Output: {1, 2, 3, 4}

empty_set = set() # Use set() to create an empty set

{1, 2, 3, 4}
```

Set Operations

```
# Adding elements
my_set.add(5)

# Removing elements
my_set.remove(2)

# Set operations
set1 = {1, 2, 3}
set2 = {3, 4, 5}

print(set1 | set2) # Union: {1, 2, 3, 4, 5}
print(set1 & set2) # Intersection: {3}
print(set1 - set2) # Difference: {1, 2}

{1, 2, 3, 4, 5}
{3}
{1, 2}
```

4. Dictionaries (dict)

A dictionary is an unordered collection of key-value pairs. It is widely used for storing structured data.

Creating a Dictionary

```
student = {"name": "Alice", "age": 21, "course": "AI"}
```

Dictionary Operations

```
# Accessing values
print(student["name"]) # Output: Alice

# Adding a new key-value pair
student["grade"] = "A"

# Removing a key-value pair
del student["age"]

# Iterating through a dictionary
for key, value in student.items():
    print(key, ":", value)
```

```
Alice
name : Alice
course : AI
grade : A
```

5. Arrays

An array is a collection of elements of the same data type stored in a contiguous memory location. While Python lists can store mixed data types, arrays in Python (from the array module) store only one type of data, making them more memory efficient.

Creating an Array

To use arrays, we need to import the array module.

```
import array

# Creating an integer array
arr = array.array('i', [1, 2, 3, 4, 5])
print(arr) # Output: array('i', [1, 2, 3, 4, 5])

array('i', [1, 2, 3, 4, 5])
```

Array Operations

```
# Accessing elements
print(arr[0]) # Output: 1

# Adding elements
arr.append(6)

# Removing elements
arr.remove(3)

# Updating an element
arr[1] = 10

# Iterating over an array
for num in arr:
    print(num)
```

```
1
1
10
4
5
6
```

For Machine Learning (ML) and numerical computing, we typically use NumPy arrays (`numpy.ndarray`) instead of the built-in array module, because NumPy supports vectorized operations, making computations much faster.

5. Conditional Statements & Loops

In Python, conditional statements and loops allow us to control the flow of execution in a program based on conditions or repetitions. These constructs are essential for decision-making and iterative operations in AI and ML applications.

1. Conditional Statements (if, elif, else)

Conditional statements execute different code blocks based on specified conditions.

Syntax

```
if condition:
    # Code executed if condition is True
elif another_condition:
    # Code executed if the previous condition was False and this one is True
else:
    # Code executed if all previous conditions were False
```

For example

```
age = 20

if age < 18:
    print("You are a minor.")
elif age >= 18 and age < 60:
    print("You are an adult.")
else:
    print("You are a senior citizen.")

You are an adult.
```

if checks the first condition; if it's True, the corresponding block executes. elif (else if) checks additional conditions if the previous if condition was False. else executes when all conditions are False.

2. Loops (for, while)

Loops are used to repeat a block of code multiple times. Python provides for loops and while loops for iteration.

a) for Loop

A for loop is used when the number of iterations is known beforehand, such as iterating over a list or range.

Syntax

```
for variable in iterable:
    # Code block executed in each iteration
```

For example

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)

apple
banana
cherry
```

Using range() in a for loop:

```
for i in range(5):  
    print(i) # Outputs: 0, 1, 2, 3, 4
```

```
0  
1  
2  
3  
4
```

b) while Loop

A while loop is used when the number of iterations is unknown in advance and depends on a condition.

Syntax:

```
while condition:  
    # Code block executed while condition is True
```

For example,

```
count = 0  
  
while count < 5:  
    print("Count:", count)  
    count += 1
```

```
Count: 0  
Count: 1  
Count: 2  
Count: 3  
Count: 4
```

3. Loop Control Statements

Python provides special statements to control loop execution:

Statement	Description
break	Exits the loop immediately.
continue	Skips the current iteration and moves to the next.
pass	Does nothing (used as a placeholder).

Example: Using break, continue, and pass.

```
for i in range(10):
    if i == 5:
        break # Exits loop when i is 5
    if i == 3:
        continue # Skips printing when i is 3
    if i == 2:
        pass # Does nothing
    print(i)
```

0
1
2
4

6. Functions & Modules

In Python, functions allow code reuse and modular programming, making programs more structured and efficient. Modules enable the use of pre-written code, reducing development time and effort.

1. Defining and Calling Functions (def)

A function is defined using the `def` keyword, followed by the function name and parentheses.

Syntax:

```
def function_name(parameters):
    # Code block
    return value # (optional)
```

For example,

```
def greet(name):
    print(f"Hello, {name}! Welcome to AI & ML.")
```

```
greet("Asad")
```

```
Hello, Asad! Welcome to AI & ML.
```

2. Function Parameters and Return Values

Functions can accept multiple parameters and return results.

Example: Function to Calculate Square of a Number

```
def square(num):  
    return num * num  
  
result = square(5)  
print("Square of 5:", result)
```

Square of 5: 25

Example: Function with Multiple Parameters (Addition of Two Numbers)

```
def add(a, b):  
    return a + b  
  
print("Sum:", add(10, 20))
```

Sum: 30

3. Using Built-in Functions

Python provides many built-in functions for common tasks. For example: Finding Maximum, Minimum, and Absolute Value

```
numbers = [-10, 25, 30, -5, 100]  
  
print("Maximum:", max(numbers))  
print("Minimum:", min(numbers))  
print("Absolute Value of -10:", abs(-10))
```

Maximum: 100
Minimum: -10
Absolute Value of -10: 10

Example: Using len(), sum(), and sorted() on a List

```
data = [3, 1, 4, 1, 5, 9]  
  
print("Length:", len(data))  
print("Sum:", sum(data))  
print("Sorted:", sorted(data))
```

Length: 6
Sum: 23
Sorted: [1, 1, 3, 4, 5, 9]

7. Importing and Using Python Libraries

Python allows importing modules to extend functionality. For example: Using the math Module

```
import math

print("Square root of 16:", math.sqrt(16))
print("Value of pi:", math.pi)
print("Factorial of 5:", math.factorial(5))
```

```
Square root of 16: 4.0
Value of pi: 3.141592653589793
Factorial of 5: 120
```

Example: Using the random Module to Generate Random Numbers

```
import random

print("Random number between 1 and 10:", random.randint(1, 10))
print("Random choice from a list:", random.choice(['AI', 'ML', 'Deep Learning']))
```

```
Random number between 1 and 10: 8
Random choice from a list: Deep Learning
```

Example: Importing numpy and Performing Matrix Multiplication

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

C = np.dot(A, B)
print("Matrix Multiplication Result:\n", C)
```

```
Matrix Multiplication Result:
[[19 22]
 [43 50]]
```

EXERCISE

1. Swap two variables without using a third variable.
2. Write a program that takes a user's name and age as input and prints a greeting message.
3. Convert a given temperature from Celsius to Fahrenheit using the formula:
$$F = (C \times 9/5) + 32$$
4. Create a list with at least 5 elements and print the first and last element.
5. Write a Python script to check whether a given number is positive, negative, or zero.
6. Take a string input from the user and print:
 - o The string in uppercase
 - o The string in reverse order
 - o The number of characters in the string
7. Create a list of five programming languages and print them one by one.
8. Create a tuple with (name, age, city) and print the values using indexing.
9. Given a list numbers = [4, 7, 2, 9, 1, 5], write a program to:
 - o Sort the list in ascending order.
 - o Find and print the maximum and minimum values.
10. Write a Python script to:
 - o Create a dictionary with student names as keys and their marks as values.
 - o Print the student with the highest marks.
11. Write a program that asks the user for a number and prints "Even" if it's even, and "Odd" otherwise.
12. Write a Python script that checks whether a given year is a leap year.
13. Develop a simple grading system:
 - o Input: User enters marks (0-100).
 - o Output: Print grades:
 - 80-100: "A"
 - 70-79: "B"
 - 60-69: "C"
 - 50- 60: "D"
 - <50: "F"

14. Write a Python script that prints numbers from 1 to 10 using a for loop.
 15. Print all even numbers between 1 and 20 using a while loop.
 16. Write a Python program that calculates the sum of all numbers from 1 to 100.
 17. Print the Fibonacci series up to n terms (user input).
 18. Write a function square(n) that returns the square of n.
 19. Write a function greet(name) that prints "Hello, [name]!".
 20. Write a function that takes a list of numbers and returns:
 - o The sum of all numbers
 - o The average value
 21. Import the math module and use it to:
 - o Find the square root of a given number.
 - o Calculate $\sin(30^\circ)$ in radians.
-

1. Swap two variables without using a third variable.

```
▶ a = 5
  b = 10
  a, b = b, a
  print("a =", a)
  print("b =", b)

... a = 10
   b = 5
```

2. Write a program that takes a user's name and age as input and prints a greeting message.

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))

print("Hello", name, "! You are", age, "years old.")

Enter your name: bushra
Enter your age: 26
Hello bushra ! You are 26 years old.
```

3. Convert a given temperature from Celsius to Fahrenheit using the formula:

$$F = (C \times 9/5) + 32$$

```
c = float(input("Enter temperature in Celsius: "))
f = (c * 9/5) + 32

print("Temperature in Fahrenheit:", f)

Enter temperature in Celsius: 100
Temperature in Fahrenheit: 212.0
```

4. Create a list with at least 5 elements and print the first and last element.

```
▶ my_list = [10, 20, 30, 40, 50]

print("First element:", my_list[0])
print("Last element:", my_list[4])

... First element: 10
   Last element: 50
```

5. Write a Python script to check whether a given number is positive, negative, or zero.

```
num = float(input("Enter a number: "))

if num > 0:
    print("Positive")
elif num < 0:
    print("Negative")
else:
    print("Zero")

Enter a number: 6
Positive
```

6. Take a string input from the user and print:

- o The string in uppercase
- o The string in reverse order
- o The number of characters in the string

```
text = input("Enter a string: ")

print("Uppercase:", text.upper())
print("Reverse:", text[::-1])
print("Number of characters:", len(text))
```

```
Enter a string: Bushra
Uppercase: BUSHRA
Reverse: arhsuB
Number of characters: 6
```

7. Create a list of five programming languages and print them one by one.

```
languages = ["Python", "Java", "C++", "JavaScript", "MATLAB"]

for lang in languages:
    print(lang)
```

```
... Python
    Java
    C++
    JavaScript
    MATLAB
```

8. Create a tuple with (name, age, city) and print the values using indexing.

```
info = ("izza", 21, "Islamabad")

print("Name:", info[0])
print("Age:", info[1])
print("City:", info[2])
```

```
Name: izza
Age: 21
City: Islamabad
```

9. Given a list numbers = [4, 7, 2, 9, 1, 5], write a program to:

- o Sort the list in ascending order.
- o Find and print the maximum and minimum values.

```
numbers = [4, 7, 2, 9, 1, 5]

numbers.sort()
print("Sorted list:", numbers)

print("Maximum:", max(numbers))
print("Minimum:", min(numbers))
```

```
... Sorted list: [1, 2, 4, 5, 7, 9]
    Maximum: 9
    Minimum: 1
```

10. Write a Python script to:

- o Create a dictionary with student names as keys and their marks as values.
- o Print the student with the highest marks.

```
students = {  
    "hamood": 85,  
    "eyad": 92,  
    "rashid": 78  
}  
  
highest = max(students, key=students.get)  
  
print("Student with highest marks:", highest)  
print("Marks:", students[highest])
```

```
Student with highest marks: eyad  
Marks: 92
```

11. Write a program that asks the user for a number and prints "Even" if it's even, and "Odd" otherwise.

```
▶ num = int(input("Enter a number: "))  
  
if num % 2 == 0:  
    print("Even")  
else:  
    print("Odd")
```

```
... Enter a number: 66  
Even
```

12. Write a Python script that checks whether a given year is a leap year:

```
year = int(input("Enter a year: "))  
  
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):  
    print("Leap Year")  
else:  
    print("Not a Leap Year")
```

```
Enter a year: 2022  
Not a Leap Year
```

13. Develop a simple grading system:

o Input: User enters marks (0-100).

o Output: Print grades:

▪ 80-100: "A"

▪ 70-79: "B"

▪ 60-69: "C"

▪ 50- 60: "D"

▪ <50: "F"

```
marks = int(input("Enter marks (0-100): "))

if marks >= 80:
    print("A")
elif marks >= 70:
    print("B")
elif marks >= 60:
    print("C")
elif marks >= 50:
    print("D")
else:
    print("F")

... Enter marks (0-100): 80
A
```

14. Write a Python script that prints numbers from 1 to 10 using a for loop.

```
for i in range(1, 11):
    print(i)

1
2
3
4
5
6
7
8
9
10
```

15. Print all even numbers between 1 and 20 using a while loop.

```
num = 2

while num <= 20:
    print(num)
    num += 2

... 2
4
6
8
10
12
14
16
18
20
```

16. Write a Python program that calculates the sum of all numbers from 1 to 100.

```
total = 0

for i in range(1, 101):
    total += i

print("Sum:", total)
```

Sum: 5050

17. Print the Fibonacci series up to n terms (user input).

```
▶ n = int(input("Enter number of terms: "))

a = 0
b = 1

for i in range(n):
    print(a)
    a, b = b, a + b
```

```
... Enter number of terms: 3
0
1
1
```

18. Write a function square(n) that returns the square of n.

```
def square(n):
    return n * n

print(square(5))
```

25

19. Write a function greet(name) that prints "Hello, [name]!". 20. Write a function that takes a list of numbers and returns:

```
def greet(name):
    print("Hello,", name + "!")

greet("Bushra")
```

Hello, Bushra!

20. Write a function that takes a list of numbers and returns:

- o The sum of all numbers
- o The average value

```
def calculate(numbers):  
    total = sum(numbers)  
    average = total / len(numbers)  
    return total, average  
  
nums = [10, 20, 30, 40]  
total, avg = calculate(nums)  
  
print("Sum:", total)  
print("Average:", avg)  
  
... Sum: 100  
    Average: 25.0
```

21. Import the math module and use it to:

- o Find the square root of a given number.
- o Calculate $\sin(30^\circ)$ in radians.

```
import math  
  
num = float(input("Enter a number: "))  
  
print("Square root:", math.sqrt(num))  
  
# sin(30 degrees) → convert to radians first  
print("sin(30) in radians=", math.sin(math.radians(30)))  
  
... Enter a number: 4  
    Square root: 2.0  
    sin(30) in radians= 0.49999999999999994
```

International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical Engineering

Artificial Intelligence & Machine Learning LAB

Lab No. 1

Hands-on Python Programming for Artificial Intelligence & Machine Learning

Name: Bushra Nazir

Reg. No.: 1071-F22

S. No.	Criteria	Allocated Percentage	Level 1 (0%)	Level 2 (25%)	Level 3 (50%)	Level 4 (75%)	Level 5 (100%)	Marks Obtained
(A) PSYCHOMOTOR (To be judged in the lab during experiment)								
1	Practical Implementation OR Connectivity/Arrangement of Equipment	5	Absent	Degree of errors in applying procedural knowledge or Degree of errors in arrangement of equipment:				
			0	With several critical errors, Incomplete and Not Neat	With few errors, Incomplete and not Neat	With some errors, Complete but not Neat	Without errors, Complete and Neat	
				1.25	2.5	3.75	5	
2	Use of Equipment OR Use of Simulation/Programming Tool	2	Absent	Use of tools, equipment and materials with:				
			0	Limited Competence	Some Competence	Considerable Competence	Good Competence	
				0.5	1	1.5	2	
3	Safety (Optional)	0	Absent	Degree of reminders to follow safety procedure with:				
			0	Rarely Follow Safety Rules	Needs Reminders	Needs minor Reminders	Follows Safety Rules	
				0	0	0	0	
SUB-TOTAL (PT)		7	SUB-TOTAL MARKS OBTAINED (PO)					
(B) COGNITIVE (To be judged on the copy of experiment submitted)								
4	Data Record, Analysis and Evaluation OR Algorithm Design	1	Absent	Data record, Analysis and Evaluation or Designed Algorithm is:				
			0	Incorrect /Incomplete	Complete with some errors	Complete with few errors	Complete and Accurate	
				0.25	0.5	0.75	1	
5	Procedural Awareness (Optional)	0	Absent	Procedural awareness of the completed task is:				
			0	Inappropriate	Appropriate with some errors	Appropriate with few errors	Appropriate	
				0	0	0	0	
SUB-TOTAL (CT)		1	SUB-TOTAL MARKS OBTAINED (CO)					
(C) AFFECTIVE (To be judged in the lab during experiment)								
6	Level of Participation & Attitude to Achieve Individual/Group Goals	2	Absent	Degree of positive interaction as an individual or within a group:				
			0	Rare interaction	Some interaction	Acceptable interaction	Good interaction	
				0.5	1	1.5	2	
SUB-TOTAL (AT)		2	SUB-TOTAL MARKS OBTAINED (AO)					

Total Marks (PT + CT + AT) :

10

Instructor

Marks Obtained (PO + CO + AO) :